

단위가 다른 주식을 하나의 자로 재는 법

pandas와 FinanceDataReader로 주가 데이터를 받아 일별 수익률 표까지 완성하는 데이터 전처리

🎯 오늘 우리가 도달할 종착점

삼성전자 5만원과 NAVER 38만원은 같은 **1,000원이 올라도 완전히 다른 의미**를 갖습니다.
엑셀의 한계를 넘어, 950개 종목을 통째로 다루고 일별 수익률 표로 표준화하는 과정을 완성합니다.

심장 한 줄 "pandas 문법을 배우는 게 아니라, 서로 다른 주식을 같은 자로 비교하는 법을 배운다."

🗺 4개 막(Act) 진행 로드맵

- 막 0** 왜 **pandas**인가 — 엑셀의 한계, for-loop vs 벡터 연산
- 막 1** 표를 채우고 세운다 — FDR 수집, Series, DataFrame 3종 세트
- 막 2** 원하는 행을 꺼내고 거른다 — loc vs iloc, Boolean 마스크
- 막 3** 세 줄 해독과 수익률 표 — shift(1), pct_change(), 원환 완결

엑셀이 멈추는 자리 — 950개 종목 수작업의 한계

"KOSPI 전체 종목 중 시가총액 상위 10개를 지금 뽑아줄 수 있나요?"

😓 엑셀 수작업의 한계

- 1 다운로드한 950개 종목 파일을 열고 스크롤
- 2 시가총액 컬럼 정렬 버튼 클릭 후 상위 10개 복사
- 3 내일 요청이 오면 **처음부터 전 과정 수동 반복**
- 4 조건 변경(상위 20개, 코스닥 포함) 시 손으로 다시 자름

⚡ 코드 한 줄의 확장성

- 1 `kospi.nlargest(10, 'Marcap')`
- 2 내일 새 데이터가 들어와도 같은 코드 **재실행 1초**
- 3 숫자 10을 20으로 바꾸면 즉시 20개 종목 슬라이싱
- 4 조건이 10단계로 늘어나도 점(.)으로 연결(체이닝)

핵심 전환 표를 눈으로 보며 마우스로 옮기는 대신, **표에게 규칙을 말로 시키는 도구**가 바로 pandas입니다.

for-loop vs pandas — 하나씩 처리 vs 통째로 벡터 연산

속도 차이 이전에, 데이터를 다루는 사고방식(패러다임) 자체가 완전히 다릅니다.



💡 통째로 다룬다는 것의 본질

- A **for-loop**: 원소를 하나 꺼내 계산하고 리스트에 append하는 과정을 사람이 지켜봐야 함.
- B **pandas**: 5개든 950개든 수식은 동일한 $s * 1.1$ 한 줄로 처리 (C-Level 벡터화).
- C **기원**: 2008년 월스트리트 헤지펀드(AQR) Wes McKinney가 금융 패널(panel) 데이터를 다루기 위해 창안.

오늘 해독할 세 줄과 주간 원환 로드맵

지금은 암호처럼 보여도 됩니다. 오늘 600번 마지막 노트에서 이 세 줄을 1:1로 해독합니다.

●●●

today_target.py (아래 단어를 클릭해 보세요)

Copy

```
1import FinanceDataReader as fdr
2df = fdr.DataReader('005930', '2026-01-02', '2026-06-30')
3df['Close'].pct_change()
```

< Interactive Ring Stepper

단어를 클릭하면 주차별 노트와 연결됩니다

[110 노트] 외부 라이브러리 호출

```
import FinanceDataReader as fdr
```

네이버·KRX 웹페이지를 직접 긁지 않고, 파이썬에서 금융 시계열 데이터를 한 줄로 불러오는 라이브러리를 단축명 fdr로 가져옵니
다.

원환 회수 규칙

100번에서 던진 코드는 600번에서 정확히 글자 단위로 일치해야 합니다.

데이터를 어디서 받나 — FinanceDataReader와 OHLCV

네이버 금융·KRX 수작업 다운로드를 단축하고 표준화된 금융 표를 생성합니다.

950+

KOSPI 상장 종목수

40

수업 실습 스냅샷 종목수

120

삼성전자 거래일수 (상반기)

6개

반환 컬럼 (OHLCV+등락률)

컬럼명	금융 용어	의미 및 특징	강의 활용 포인트
Open	시가	그날 처음 거래된 가격	장 시작 시점 시장 반응
High / Low	고가 / 저가	그날 가장 높았던 / 낮았던 가격	하루 중 가격 변동폭(노이즈)
Close	종가	그날 마지막 체결 가격	수익률 계산의 기준점 (오늘의 핵심)
Volume	거래량	하루 동안 거래된 주식 수	시장 유동성 및 신뢰도 판단
Change	전일 대비 등락률	FDR이 제공하는 단순 변동률	600번 pct_change()와 일치 검증용

종목코드는 숫자가 아니라 문자열('005930')이다

dtype 옵션 없이 읽는 순간 선행 0이 영구히 소실되어 종목 조회가 불가능해집니다.

dtype_trap_check.py
Copy

```

1# ❌ 잘못된 방식: 선행 0 유실
2wrong = pd.read_csv('data/kospi_listing.csv')
3print(wrong['Code'].head(1))
4# 출력: 5930 (int64) ➡ 삼성전자 조회 실패!
5
6# ✅ 올바른 방식: 문자열 강제 지정
7kospi = pd.read_csv('data/kospi_listing.csv',
8                    dtype={'Code': str})
9print(kospi['Code'].head(1))
10# 출력: '005930' (str) ➡ 정상 조회!

```

⚠️ CRITICAL FAILURE: 선행 0 소실 방어

dtype={'Code': str} 필수

🚨 식별자(Identifier) vs 계산 대상(Value)

- 1

식별자: 종목코드, 전화번호, 우편번호, 주민번호는 더하거나 곱할 수 없는 고유 라벨입니다.
- 2

파이썬 정수 변환의 함정: 정수 005930은 컴퓨터 메모리에서 앞자리 0을 무의미한 숫자로 보고 5930으로 저장합니다.
- 3

철칙: 금융 데이터를 CSV/DB에서 읽을 때 종목코드는 반드시 dtype={'Code': str}를 꼭 박습니다.

실무 경고: '5930'으로 KRX에 조회하면 존재하지 않는 코드이거나 전혀 다른 상장 폐지 기업을 가리킵니다.

번호표만 있는 리스트 vs 이름표(인덱스) 달린 Series

prices[2] = 60,000원은 언제 데이터인가? 값 옆에 날짜/라벨이 붙어야 금융 데이터가 됩니다.

💡 ELI5 직관 비유 #1

목욕탕 번호표 vs 이름표 달린 사물함

파이썬 리스트는 [0], [1], [2] 번호표만 있어서 "2번 사물함에 든 게 누구 것인지"를 사람이 따로 외워야 합니다.

pandas Series는 값에 ['1월', '2월', '3월'] 또는 ['005930', '035420'] 이름표(Index)를 영구히 부착합니다.

📦 파이썬 리스트 (위치만 존재)

prices = [50000, 55000, 60000]

prices[2] ➡ 60000 (이게 언제 값인가?)

데이터가 950개로 늘어나면 몇 번째인지 기억할 수 없습니다.

📋 pandas Series (인덱스 결합)

s = pd.Series(prices, index=['1월', '2월', '3월'])

s['3월'] ➡ 60000 (3월 증가 확정!)

인덱스를 종목코드나 날짜로 쓰면 직접 이름으로 질의합니다.

for-loop 5줄이 1줄이 되는 벡터 연산

$s * 1.1$ 과 $(s - \text{buy_price}) / \text{buy_price}$ — 원소가 늘어나도 코드는 늘어나지 않습니다.

vectorized_calc.pyCopy

```
1s = pd.Series([50000, 55000, 60000, 58000, 62000],
2              index=['1월', '2월', '3월', '4월', '5월'])
3
4# 10% 상승 시나리오 일괄 적용 (1줄)
5print(s * 1.1)
6
7# 매수가 50,000원 대비 수익률 계산 (1줄)
8buy_price = 50000
9returns = (s - buy_price) / buy_price
```

1월: 0.00 | 2월: 0.10 | 3월: 0.20 | 4월: 0.16 | 5월: 0.24dtype: float64

Series 주요 통계량 요약

메서드	하는 일	실행 결과
s.max()	최댓값 단일 반환	62,000원
s.mean()	평균값 단일 반환	57,000.0원
s.describe()	개수·평균·표준편차·사분위수 종합	8대 통계표 반환

원환 회수 포인트 100번에서 보았던 for-loop 5줄의 복잡성이 여기서 완벽히 한 줄로 압축되었습니다.

엑셀 시트 한 장 — DataFrame의 개념 모델

Series 여러 개를 옆으로 나란히 붙인 2차원 표 (행=종목 × 열=속성)

🧩 Series 결합 모델

종목코드, 종목명, 시가총액이 흩어져 있으면 관리가 불가능합니다. 공통 인덱스를 기준으로 하나로 결합합니다.

행 (인덱스)	Code	Name	Marcap(원)
0	'005930'	삼성전자	1,464,492,791,304,000
1	'000660'	SK하이닉스	1,178,284,184,745,000
2	'005935'	삼성전자우	148,839,858,156,500

🛠️ 생성 방식: 컬럼 중심 vs 행 중심

- 컬럼


```
pd.DataFrame({'ticker': [...], 'Marcap': [...]})
```

 컬럼 단위로 데이터를 가공할 때 직관적입니다.
- 행


```
pd.DataFrame([{'ticker': '005930', ...}, ...])
```

 API 응답 등 JSON 레코드 단위로 수신할 때 편리합니다.

결론 두 방식 모두 최종적으로 메모리 상에서 완벽히 동일한 DataFrame으로 귀결됩니다.

표를 받으면 가장 먼저 하는 탐색 3종 세트

데이터를 안 보고 코드부터 쓰지 마세요. shape ➡ columns ➡ dtypes 확인 습관

dataframe_inspect.py Copy

```
1df = pd.read_csv('data/kospi_listing.csv', dtype={'Code': str})
2print(df.shape)
3# 출력: (40, 5) ➡ 40종목, 5개 속성
4print(df.columns)
5# 출력: ['Code', 'Name', 'Market', 'Sector', 'Marcap']
6print(df.dtypes)
7# Code: str(문자), Marcap: int64(숫자)
8print(df.head(3))
```

Check 1: Shape | Check 2: Cols | Check 3: Types

head(3)로 값 최종 육안 검증

🔍 무엇을 판단하기 위해 찍는가?

메서드	반환형	핵심 판단 목적
df.shape	(행, 열)	데이터 누락 없이 온전한 크기인가?
df.columns	Index 리스트	내가 계산에 쓸 컬럼명이 정확한가?
df.dtypes	열별 타입	Code가 문자인가? Marcap이 숫자인가?
df.head(3)	앞 3행	실제 값이 기대한 모양인가?

대괄호의 갈림길 — Series vs DataFrame

df['Marcap'](Series)과 df[['Name', 'Marcap']](DataFrame) — 대괄호 개수로 타입이 갈립니다.

bracket_syntax_split.py

Copy

```

1# 단일 대괄호: 1차원 Series 반환
2s = df['Marcap']
3print(type(s)) # <class 'pandas.Series'>
4print(s.mean()) # 200번의 Series 메서드 그대로 작동!
5
6# 이중 대괄호: 2차원 DataFrame 유지
7sub_df = df[['Name', 'Marcap']]
8print(type(sub_df)) # <class 'pandas.DataFrame'>
            
```

df['Col']

➡

Series

df[['Col1', 'Col2']]

➡

DataFrame

🔗 200번과 300번의 유기적 결합

- 1 DataFrame은 Series 여러 개를 옆으로 붙인 것이므로, 컬럼 하나를 뽑으면 다시 **Series**가 됩니다.
- 2 따라서 200번에서 배운 `.mean()`, `.max()`, `.describe()`가 `df['Marcap']`에 100% 동일하게 적용됩니다.
- 3 새로운 메서드를 외우는 것이 아니라, 이미 배운 Series 도구를 꺼내 쓰는 것입니다.

FD-02-300__dataframe

Slide 11

이름표로 꺼내기(loc) vs 물리적 번호로 꺼내기(iloc)

L = Label(이름표 검색), i = integer(정수 순번 세기) — 같은 행을 가리켜도 기준이 다릅니다.

💡 **ELI5** 직관 비유 #2

도서관 책 제목 검색 vs 1번 서가 맨 윗줄 책 꺼내기

df.loc['005930']은 '삼성전자'라는 **이름표(Label)**를 찾아갑니다. 서가 순서가 바뀌어도 항상 같은 책이 나옵니다.

df.iloc[0]은 서가 맨 앞의 **물리적 위치(0번)**를 클릭합니다. 맨 앞에 새 책이 꽂히면 다른 책이 나옵니다.

📄 **.loc[] — Label 기준 검색**

```
df_indexed = df.set_index('Code')
df_indexed.loc['005930', 'Name'] ➡ 삼성전자
```

인덱스 이름표가 일치하는 행을 정확히 반환합니다.

1234

.iloc[] — Integer Position 기준

```
df_indexed.iloc[0]['Name'] ➡ 삼성전자
(현재 0번째 행이 삼성전자이므로 지금은 결과가 같습니다)
```

표의 물리적 순서(0번째, 1번째)를 정수로 세어 접근합니다.

행이 추가되면 무슨 일이 일어나는가? (loc 불변 vs iloc 변동)

표의 맨 위에 새 행이 추가되는 순간, iloc[0]의 주인이 바뀝니다.

insert_row_test.py

Copy

```

1# 맨 위에 신규 종목 '999999' 추가
2new_row = pd.DataFrame([{'Name': '신규상장A'}], index=['999999'])
3df2 = pd.concat([new_row, df_indexed])
4
5print(df2.loc['005930', 'Name'])
6# ➡ 출력: 삼성전자 (라벨 검색 불변!)
7print(df2.iloc[0]['Name'])
8# ➡ 출력: 신규상장A (0번째 주인이 바뀜!)
            
```

loc['005930'] ➡ 삼성전자 (불변)

iloc[0] ➡ 신규상장A (변동)

⚖️ 도구 선택 기준

- loc** 특정 종목/특정 날짜의 데이터를 정확히 조회할 때 사용 (데이터 순서가 바뀌어도 안전).
- iloc** 상위 N개 행, 최근 5일치 데이터 등 물리적 위치나 슬라이싱이 목적일 때 사용.

범위를 자를 때 — loc 끝 포함 vs iloc 끝 미포함

loc['005930':'005935']는 3행 반환, iloc[0:2]는 파이썬 슬라이싱 규칙 그대로 2행 반환

loc['005930':'005935'] ➡ 3행 반환

이름표가 붙은 구간 전체를 포함 [시작, 끝]

Code (Index)	Name	포함 여부
'005930'	삼성전자	시작 포함
'000660'	SK하이닉스	중간 포함
'005935'	삼성전자우	끝 라벨 포함!

iloc[0:2] ➡ 2행 반환

파이썬 리스트 규칙과 동일한 반열림 구간 [시작, 끝)

위치 (Index)	Name	포함 여부
0	삼성전자	0번 포함
1	SK하이닉스	1번 포함
2	삼성전자우	2번 미포함 (제외)

주의 과거에 쓰이던 `df.ix[ ]`는 pandas 0.20에서 사용 중단 예고된 뒤 pandas 1.0(2020년)에서 완전히 제거되었으므로 절대 사용하지 않습니다.

조건으로 행 꺼내기 — Boolean 마스크의 원리

조건식을 컬럼에 걸면 True/False Series가 나오고, df[mask]가 True 행만 통과시킵니다.

- 전체 40개 보기
- 시총 50조 이상 (13개)
- 50조~150조 (11개)

Code	Name	Marcap(조원)	Mask (T/F)
'005930'	삼성전자	1464조	True
'000660'	SK하이닉스	1178조	True
'005935'	삼성전자우	148조	True
'034730'	SK스퀘어	129조	True
'009150'	삼성전기	104조	True

💡 대괄호 안에서 일어나는 일

```
mask = df['Marcap'] >= 50_000_000_000_000
result = df[mask] # 40행 ➡ 13행
```

순서를 뒤집어 df[df['Marcap'] >= 50_000_000_000_000]부터 외우면 대괄호 안에서 무슨 일이 벌어지는지 모릅니다. 마스크 변수가 True/False Series라는 것을 먼저 눈으로 봅니다.

실측 결과

 시가총액 50조 이상 조건: 40개 중 13개 종목 통과 (True).

복합 조건에서 괄호가 없으면 터지는 이유

&와 |는 비교 연산자보다 우선순위가 높습니다 — (cond1) & (cond2) 괄호 필수

boolean_compound_trap.pyCopy

```
1# ❌ 괄호 없이 & 쓴 경우: 런타임 에러 발생!
2df[df['Marcap'] >= 50_000_000_000_000 & df['Marcap'] <= 150_000_000_000_000]
3# ➡ ValueError: The truth value of a Series is ambiguous.
4
5# ✅ 조건마다 괄호로 완벽히 격리
6cond1 = (df['Marcap'] >= 50_000_000_000_000)
7cond2 = (df['Marcap'] <= 150_000_000_000_000)
8result = df[cond1 & cond2] # 11행 반환!
```

⚠️ ValueError 방어반드시 (cond1) & (cond2) 괄호 작성

🚩 왜 아는 학생이 더 자주 틀리는가?

파이썬 기초에서 and, or를 능숙하게 쓰던 사람일수록 자연스럽게 괄호 없이 &를 적습니다.
비트 연산자 &는 >=보다 먼저 계산되므로 파이썬이 50조 & df['Marcap']을 먼저 묶어버립니다.

규칙: pandas에서 &(AND)와 |(OR)를 쓸 때는 무조건 조건마다 소괄호 ()를 씁니다.

정렬과 메서드 체이닝 — `sort_values`와 `nlargest`

필터 ➡ 정렬 ➡ 컬럼 선택이 점(.) 하나로 이어지는 파이프라인

method_chaining.pyCopy

```
1# 정렬 후 상위 10개 슬라이싱 체이닝
2top10 = (kospi.sort_values('Marcap', ascending=False)
3         [['Name', 'Marcap']]
4         .head(10))
5
6# 정렬+자르기 단축: nlargest()
7top10_fast = kospi.nlargest(10, 'Marcap')[['Name', 'Marcap']]
```

1위 삼성전자 | 2위 SK하이닉스 | 3위 삼성전자우시총 Top 10 추출

🏆 KOSPI 시가총액 상위 5개사

순위	종목명	시가총액 (원)
1	삼성전자	1,464,492,791,304,000
2	SK하이닉스	1,178,284,184,745,000
3	삼성전자우	148,839,858,156,500
4	SK스퀘어	129,549,366,036,000
5	삼성전기	104,571,174,400,000

서사 전환

시총 상위 10개 추출은 종착점이 아니라, 수익률 표를 만들기 위한 중간 관문입니다.

100번의 세 줄, 한 단어씩 완전 해독

오늘 시작할 때 보았던 그 암호 같던 코드가 완벽히 해독되어 회수됩니다.

코드 구문	배운 노트	해독된 금융/데이터 개념
<code>import FinanceDataReader as fdr</code>	110	KRX/글로벌 금융 시계열 데이터를 파이썬 표로 수집하는 라이브러리 호출
<code>df = fdr.DataReader('005930', ...)</code>	110 + 300	종목코드(문자열)와 기간을 전달받아 날짜 인덱스가 붙은 120행 6열 DataFrame 생성
<code>df['Close']</code>	300	2차원 DataFrame에서 종가(Close) 컬럼을 대괄호 하나로 꺼내어 1차원 Series 로 변환
<code>.pct_change()</code>	600 (지금!)	단위가 다른 종가를 일별 단순수익률(((오늘-어제)/어제))로 변환하는 최종 연산자

원환 완결

100번에서 시작된 긴장이 600번에서 완벽히 해소되었습니다. 이제 가격이 아닌 수익률의 세계로 진입합니다.

왜 가격으로는 비교가 안 되나 — 1,000원 상승의 서로 다른 무게

삼성전자와 NAVER가 똑같이 1,000원 올랐다? 절대 금액은 같지만 수익률은 8배 차이납니다.

💡 **ELI5 직관 비유 #4**
키 50cm 아이의 1cm 성장 vs 키 180cm 어른의 1cm 성장
 같은 1cm라도 50cm 아이에게는 +2%의 거대한 성장이지만, 180cm 어른에게는 미미한 변동입니다.
 주가도 마찬가지입니다. **출발점이 다른 자산은 절대 가격으로 비교할 수 없습니다.**

절대 가격 변화 보기
 수익률 기준 보기 (하나의 자)

[절대 가격 기준] 똑같이 +1,000원 상승한 경우
 삼성전자 (50,000원 ➡ 51,000원) +1,000원
 NAVER (380,000원 ➡ 381,000원) +1,000원
 착시: 두 막대의 길이가 완전히 같아 보여 같은 성과로 오판하게 됩니다.

📊 실제 상반기 1월 데이터 실측 비교

종목	1/8 종가	1/9 종가	절대 변화	실제 수익률
삼성전자	61,100원	62,100원	+1,000원	+1.64%
NAVER	383,400원	387,400원	+4,000원	+1.04%

심장 한 줄
 단위가 다른 주식을 하나의 공통된 자로 재는 방법이 바로 **수익률**입니다.

어제 값을 오늘 행으로 — shift(1)과 첫 행 NaN

pct_change()를 쓰기 전, 전날 종가를 오늘 옆 칸으로 밀어 놓는 과정을 시각화합니다.

원본 종가 (Close)

shift(1) 적용 (Close_어제)

Date	Close	Close_어제 (shift(1))
2026-01-02	62,000	NaN (어제 없음)
2026-01-05	61,800	62,000
2026-01-06	61,600	61,800
2026-01-07	60,900	61,600
2026-01-08	61,100	60,900

💡 첫 행 NaN은 오류가 아니라 사실이다

- 11월 2일은 분석 기간의 첫 번째 날이므로 '어제(전날)'가 존재하지 않습니다.
- 따라서 첫 행의 close_어제와 Return이 비어 있는(NaN) 것은 정상적인 사실입니다.
- 이 그림을 먼저 이해해야 다음 절에서 만날 NaN 처리 규칙이 자연스럽게 납득됩니다.

일별 수익률 표 완성 — pct_change()와 NaN 정책

수익률 = (오늘 증가 - 어제 증가) / 어제 증가 ➡ 첫 행 NaN을 0으로 채우지 않습니다.

returns_pipeline_final.py

Copy

```

1samsung = pd.read_csv('data/samsung_005930.csv', index_col='Date')
2samsung['Return'] = samsung['Close'].pct_change()
3print(samsung[['Close', 'Return']].head(4))
4
5# ⚠ 절대 0으로 채우지 않는다 (fillna(0) 금지!)
6clean_returns = samsung['Return'].dropna()
            
```

2026-01-02: NaN

|

2026-01-05: -0.003226

일별 수익률 표 완성

🚩 NaN을 0으로 채우면 왜 안 되는가?

- 1

왜곡 발생: 1월 2일의 수익률을 0%로 강제 채우면, 실제로는 알 수 없는 첫날에 "수익률 0%로 거래를 마쳤다"는 가짜 데이터가 주입됩니다.
- 2

누적수익률 오차: 향후 복리 계산(Cumulative Returns) 시 기간 전체의 복리 성과가 어긋납니다.
- 3

올바른 처리: dropna()로 첫 행을 날리거나, NaN 상태 그대로 보존합니다.

이번 주 도착점과 다음 연결

오늘 만든 일별 수익률 표는 12주차 기말 포트폴리오 프로젝트의 첫 조각입니다.

🏆 오늘 우리가 달성한 4대 역량

- 1 FDR로 상장종목/주가 수집 및 `dtype={'Code': str}` 방어
- 2 Series 인덱스 이름표와 1줄 벡터 연산(`s * 1.1`) 체화
- 3 `loc` vs `iloc` 구분 및 Boolean 복합 조건 괄호 필수 방어
- 4 100번 세 줄 완전 해독 및 `pct_change()` 일별 수익률 표 완성

🚀 다음 주차 예고 (Week 03)

- W03 금융 시계열 파이프라인 & 로컬 SQLite DB 구축
- 질문 "API에서 매번 받지 않고 내 컴퓨터 DB에 어떻게 저장해 쓰나?"
- 기술 `DatetimeIndex`, 비동기 거래일 결측 처리, `df.to_sql()`

실전 연결 증권사/자산운용사 신입의 첫 업무가 바로 오늘 만든 데이터 표를 시스템에 적재하는 일입니다.